

Express.js + MongoDB CRUD Operations (Part 3)

Add, Update, and Delete Data Using Mongoose

This tutorial is the **third part** of a four-part series on building a **CRUD application using Node.js, Express.js, MongoDB, and Mongoose**.

Using Mongoose Documentation

The tutorial references the official **Mongoose documentation**:

- Website: mongoosejs.com
 - Important section: **Models**
 - Key methods used:
 - `findById`
 - `create`
 - `findByIdAndUpdate`
 - `findByIdAndDelete`
-

Project Setup Overview

- Express.js application with EJS templating
 - MongoDB database connected using Mongoose
 - A `contacts` collection containing fields:
 - `first_name`
 - `last_name`
 - `email`
 - `phone`
 - `address`
 - Data displayed on the home page
 - Options for:
 - View single contact
 - Add contact
 - Update contact
 - Delete contact
-

1. Adding New Data (Create)

Step 1: Create a GET Route for Add Contact Page

```
app.get('/add-contact', (req, res) => {  
  res.render('add-contact');  
});
```

Step 2: Wrap the Form in EJS with `<form>`

In `add-contact.ejs`:

```
<form action="/add-contact" method="POST">  
  <!-- input fields -->  
</form>
```

- `method="POST"` is used for secure data submission
- Form fields use `name` attributes matching database field names

Step 3: Enable Middleware to Read Form Data

```
app.use(express.urlencoded({ extended: true }));
```

This middleware is required to access `req.body`.

Step 4: Handle Form Submission (POST Route)

```
app.post('/add-contact', async (req, res) => {  
  await Contact.create(req.body);  
  res.redirect('/');  
});
```

Why `create()`?

- Automatically maps form fields to database fields
- Works only if **form field names match schema fields**
- Cleaner and shorter than `insertOne`

Result

- Data is stored in MongoDB

- User is redirected to the home page
- Newly added contact appears immediately

2. Updating Existing Data (Update)

Step 1: Open Update Page Using Contact ID

```
app.get('/update-contact/:id', async (req, res) => {  
  const contact = await Contact.findById(req.params.id);  
  res.render('update-contact', { contact });  
});
```

Step 2: Pre-Fill Form with Existing Data

In `update-contact.ejs`:

```
<input type="text" name="first_name" value="<%= contact.first_name %>">  
<input type="email" name="email" value="<%= contact.email %>">
```

For `<textarea>`:

```
<textarea name="address"><%= contact.address %></textarea>
```

Step 3: Add Form Action with Contact ID

```
<form action="/update-contact/<%= contact._id %>" method="POST">
```

Step 4: Handle Update Submission

```
app.post('/update-contact/:id', async (req, res) => {  
  await Contact.findByIdAndUpdate(req.params.id, req.body);  
  res.redirect('/');  
});
```

Alternative: When Field Names Do NOT Match

```
await Contact.findByIdAndUpdate(req.params.id, {
  first_name: req.body.fname,
  last_name: req.body.lname,
  email: req.body.email,
  phone: req.body.phone,
  address: req.body.address
});
```

Use this approach when form field names differ from schema fields.

Result

- Selected contact is updated
 - Changes reflect immediately on home and view pages
-

3. Deleting Data (Delete)

Step 1: Pass Contact ID in Delete Button

In EJS home page:

```
<a href="/delete-contact/<%= contact._id %>">Delete</a>
```

Step 2: Handle Delete Route

```
app.get('/delete-contact/:id', async (req, res) => {
  await Contact.findByIdAndDelete(req.params.id);
  res.redirect('/');
});
```

Why `findByIdAndDelete()`?

- Simplest method for deleting by ID
 - No need to specify fields or conditions
-

Result

- Contact is removed from MongoDB
- Page reloads automatically
- Updated list is displayed

Key Best Practices Highlighted

- Use **POST** for form submissions
 - Always use **async/await** with database operations
 - Redirect users after Create, Update, Delete actions
 - Match form field names with schema fields for cleaner code
 - Prefer Mongoose helper methods over raw MongoDB queries
-

What's Coming Next

In the next tutorial:

- Refactor code into **MVC architecture**
 - Implement **error handling**
 - Improve **data ordering and indexing**
 - Clean and optimize the project structure
-

Summary

This tutorial demonstrated how to:

- Insert form data into MongoDB
- Update records using ID
- Delete records using ID
- Use Mongoose's built-in methods efficiently
- Integrate forms with Express routes and EJS templates